

Post-Quantum Authentication Latency in TLS 1.3: A Validated Model and the Limits of Deferred Signature Verification

Tahera Begum Abdul^{a,*}, K. Venkata Ramana^a

^a*Department of Computer Science and Systems Engineering, Andhra University, Visakhapatnam, Andhra Pradesh, India*

Abstract

Hybrid post-quantum (PQ) certificates make TLS 1.3 handshakes slower, but it is unclear how much of the cost is computation and how much is network, and which mitigations help. We develop a parameter-free latency model for PQ authentication in TLS 1.3 that separates processing, signature verification on the critical path, and round trips induced by the TCP initial congestion window, and we validate it with more than 46,000 individually timed handshakes in OpenSSL 3.5 using five hybrid and classical schemes (ECDSA, ML-DSA-44/65, FN-DSA-512, SLH-DSA-128f), three-certificate chains, X25519MLKEM768 key exchange, and client and server on separate network stacks and cores over an emulated 1500-byte-MTU link. The model predicts the number of round trips in all 68 tested configurations of scheme, RTT (20–150 ms), initial window and congestion controller, and the median latency with a mean absolute error of 2.2%. Round trips dominate: a single extra round trip, incurred when a chain’s first flight exceeds the initial window (as with ML-DSA-65), costs more than all PQ computation, and raising the server’s initial window from 10 to 40 segments cuts SLH-DSA handshakes at 150 ms RTT from 454 to 165 ms. We also specify and evaluate deferred asynchronous PQ verification (DAPV), a client-only change that completes the handshake after the classical check and verifies the PQ components in background workers under a precisely defined provisional state. Its latency gain equals the verification work removed from the critical path minus a dispatch cost ($R^2 = 0.999$), with a break-even point of about 0.14 ms of verification work per handshake. Lattice-based schemes on an AVX2 CPU fall below it and gain nothing, whereas clients without vector instructions gain 9% (ML-DSA) to 26% (SLH-DSA), and slower verifiers up to 65%. The provisional window lasts 0.08 ms at the median (0.42 ms at the 99th percentile), deferral does not raise server throughput, and 3% packet loss more than doubles the 90th-percentile latency of ML-DSA and FN-DSA handshakes but not of classical ones. We derive deployment guidance and release all code and data.

Keywords: Post-quantum cryptography, TLS 1.3, Hybrid authentication, Performance model, TCP initial congestion window, Deferred verification, ML-DSA, SLH-DSA

1. Introduction

1.1. Why post-quantum authentication, and why now

The public-key algorithms used by TLS today—ECDH/X25519 for key exchange and RSA or ECDSA for authentication—rely on the hardness of integer factorisation and discrete logarithms, both of which are solved efficiently by Shor’s algorithm [1] on a sufficiently large quantum computer. The two uses of public-key cryptography in TLS, however, face quantum threats of a different nature and urgency.

Confidentiality is threatened today by the “harvest now, decrypt later” (HNDL) strategy [2]: an adversary records encrypted sessions now and recovers their key-exchange secrets once a cryptographically relevant quantum computer (CRQC) exists. HNDL is countered by post-quantum or hybrid key exchange such as X25519MLKEM768 [3, 4], which major browsers and content delivery networks have already deployed [5]. *Authentication* is not vulnerable to HNDL: a signature that was verified in a past handshake cannot be retroactively forged to affect that handshake. The quantum threat to authentication is instead *active* impersonation once a CRQC can forge server signatures in real time. Why, then, migrate authentication now? Because the systems that carry authentication—certificate authorities, root programmes, hardware security modules, device firmware with pinned trust anchors, and certificates with multi-year validity—take many years to change. By Mosca’s inequality [6], if the migration time plus the lifetime of deployed trust anchors exceeds the time until a CRQC appears, migration must begin now. National agencies have therefore published timelines covering both key establishment and signatures [7, 8, 9], following NIST’s standardisation of ML-DSA [10], SLH-DSA [11] and the forthcoming FN-DSA [12].

*Corresponding author

Email address: taherabegum.abdul@gmail.com (Tahera Begum Abdul)

This paper is concerned only with the authentication side. We state this explicitly because the two problems are often conflated: nothing in this paper mitigates HNDL, and our experiments are therefore designed so that the key-exchange configuration is held fixed across all measured variants (Section 6.4).

1.2. Hybrid authentication and its costs

During the transition, a server may authenticate with a classical signature, a PQ signature, or both. *Hybrid* authentication—in which the client accepts the server only if both a classical and a PQ signature verify—is recommended by several agencies as a hedge: the PQ schemes are young, and some candidates (e.g., Rainbow and SIKE) were broken late in the NIST process, while ECDSA remains secure against classical adversaries. Under an AND-combination, an attacker must break *both* components [13, 14]. Hybrid authentication is thus a security measure; it is *not* an interoperability mechanism. A hybrid credential in a single certificate (a *composite* key [15]) cannot be verified by a client that does not understand the composite algorithm; interoperability with such clients requires that the server select among several certificate chains using the `signature_algorithms` extension, or that the certificate carry the PQ key in a non-critical extension that legacy clients ignore [13]. Section 2.2 reviews these options and explains which of them DAPV applies to.

The cost of PQ authentication is dominated by size. An ML-DSA-44 public key and signature are 1312 and 2420 bytes, respectively, against 65 and at most 72 bytes for ECDSA P-256 (Table 1). A hybrid server flight carries at least two PQ signatures (one in the leaf certificate and one in `CertificateVerify`) plus one PQ public key; with an intermediate certificate, it carries one more of each. When the flight exceeds the TCP initial congestion window (typically ten segments, about 14.6 KB [16]), an additional round trip is required, which is the principal cause of the latency increases and throughput reductions reported in the literature [17, 18, 19, 20]. Size-reduction techniques (certificate compression, intermediate-certificate suppression [21, 22], mixed certificate chains [23], KEM-based authentication [24, 25]) address that component.

The second, smaller, component is computation. A hybrid client performs one classical and one PQ verification for `CertificateVerify`, and one of each for every certificate signature in the chain, all synchronously on the handshake critical path. DAPV targets this component *only*: it does not reduce the number of bytes on the wire and does not remove any fragmentation- or congestion-window-induced round trip.

1.3. Research questions

PQ signature verification is fast on processors with vector extensions—we measure 37–59 μ s per ML-DSA or FN-DSA verification with AVX2—yet hybrid authentication adds more than a millisecond to a handshake even without any network delay, and whole round trips with larger chains. Operators choosing schemes, chain layouts and TCP settings, and implementers deciding where to spend engineering effort, need to know which part of this cost comes from computation and which from the network, and which mitigations actually work. We ask:

RQ1

Can the latency of hybrid PQ authentication in TLS 1.3 be *predicted* from a few measurable quantities, and which of them dominate?

RQ2

When does deferring PQ signature verification past handshake completion—a purely client-side change—reduce latency, and by how much?

RQ3

What security exposure does deferral create, and how can it be bounded?

RQ4

How do network conditions and TCP configuration (initial congestion window, congestion control, loss) change the picture?

To answer RQ2 and RQ3 we specify and implement *Deferred Asynchronous PQ Verification* (DAPV): the classical component of a hybrid signature is verified synchronously, and the PQ components—already bound into the handshake transcript—are verified by background workers, after which the session is promoted from a PROVISIONAL to a FULLY_AUTHENTICATED state or terminated.

1.4. Summary of findings

F1 Round trips dominate, and a parameter-free model predicts them. The number of round trips follows from the size of the server’s first flight and the initial congestion window alone (Equation (1)); the model predicts it correctly for all 68 measured configurations, with a mean absolute latency error of 2.2%. One extra round trip (ML-DSA-65, three-certificate chain) costs more than all PQ computation combined.

F2 Deferral pays off only above a break-even point. Its gain equals the verification work removed from the critical path minus a dispatch cost of ≈ 0.14 ms ($R^2 = 0.999$). On an AVX2 server CPU the lattice schemes fall just below this point and gain nothing; without vector instructions ML-DSA gains 9%, SLH-DSA-128f gains 13–26%, and with millisecond-scale verification deferral cuts latency by up to 65% and also completes full authentication sooner.

F3 The provisional window is short and boundable. With a bounded worker pool it lasts 0.08 ms at the median and 0.42 ms at the 99th percentile for ML-DSA-44; the resulting exposure is bounded by an application-layer data cap.

F4 Deferral does not raise server throughput. The server’s work is unchanged, in both separate and co-located CPU placements.

F5 TCP configuration matters more than verification. Raising the initial congestion window to 40 segments removes the extra round trips of large chains (e.g., from 454 to 165 ms for SLH-DSA-128f at 150 ms RTT); BBR’s pacing, however, retains part of their cost, and under 3% loss the 90th-percentile latency of ML-DSA-44 and FN-DSA-512 handshakes more than doubles while that of classical P-256 does not.

1.5. Contributions

- (1) A **latency model** for PQ authentication in TLS 1.3 that separates processing, critical-path verification and congestion-window-induced round trips, with no fitted parameters, and its validation across five schemes, five RTTs, three initial windows and two congestion controllers (Sections 5 and 7.1).
- (2) The **specification of deferred PQ verification**: the semantics of the provisional state (permitted data, session tickets and resumption, alerts), transcript binding, and an application interface (Section 4).
- (3) An **open implementation** in OpenSSL 3.5 with the OQS provider, and a reproducible testbed with client and server on separate network stacks and cores, a 1500-byte-MTU emulated link with loss, and three-certificate hybrid chains with X25519MLKEM768 (Section 6).
- (4) A **systematic evaluation** of more than 46,000 individually timed handshakes that quantifies the break-even point of deferral, its security exposure, and the effect of initial window, congestion control and loss (Section 7), and **deployment guidance** derived from it (Section 9).

1.6. Scope and limitations

DAPV requires a classical component to provide the synchronous anchor for the provisional state; PQ-only deployments are out of scope. During the provisional window the session has classical-strength authentication only, so DAPV is unsuitable wherever even a bounded interval of classical-only authentication is unacceptable. Our testbed separates client and server by network namespaces and disjoint CPU cores on one virtual machine rather than by physical hosts; the implications are discussed in Section 7.7.

Section 2 gives the background on PQ signatures, hybrid constructions and TLS 1.3 authentication, and Section 3 reviews related work. Section 4 presents the design, Section 6 the experimental setup and Section 7 the results. Section 8 analyses security, Section 9 discusses deployment and Section 11 concludes.

2. Background

2.1. Post-quantum signature schemes

A signature scheme is *post-quantum* if its security rests on problems for which no efficient quantum algorithm is known, as opposed to factoring and discrete logarithms. NIST has standardised three families:

- **ML-DSA** (FIPS 204, formerly CRYSTALS-Dilithium) [10]: a Fiat–Shamir-with-aborts scheme based on the Module-LWE and Module-SIS lattice problems. Signing and verification use only integer arithmetic and the NTT, are simple to implement in constant time and vectorise well (AVX2 implementations are part of the reference package [26]). Signatures and keys are comparatively large.
- **FN-DSA** (FIPS 206, formerly Falcon; draft at the time of our experiments) [12]: a hash-and-sign scheme over NTRU lattices. It has the smallest combined public key and signature of the standardised schemes and fast verification, but signing requires floating-point Gaussian sampling, which complicates constant-time implementation.
- **SLH-DSA** (FIPS 205, formerly SPHINCS+) [11]: a stateless hash-based scheme whose security relies only on properties of the hash function. It has tiny public keys but signatures of 7.8–50 KB and markedly slower signing and verification.

Table 1: Sizes (bytes) of the objects that appear in a TLS 1.3 handshake for the algorithms discussed. Values from the respective standards.

Algorithm	Security category	Public key	Signature / ciphertext
ECDSA P-256	classical	65	≤ 72
FN-DSA-512 (Falcon)	1	897	≈ 666
ML-DSA-44	2	1312	2420
ML-DSA-65	3	1952	3309
SLH-DSA-SHA2-128s	1	32	7856
X25519MLKEM768 (key exchange)	3	1216 (client share)	1120 (server share)

Choosing among them is a trade-off between size (which drives network cost), verification speed (which drives client CPU cost), signing speed and implementation risk (which drive server cost and side-channel exposure), and the conservativeness of the security assumption. For TLS server authentication, where every handshake transmits the leaf public key and at least two signatures, ML-DSA-44 and FN-DSA-512 are the most commonly evaluated options; we use both. Table 1 lists the object sizes relevant to this paper.

2.2. Hybrid signature constructions

A hybrid (or *multi-algorithm*) authentication combines a classical and a PQ signature so that security holds while at least one of them is unbroken [13, 14, 27]. Four constructions are relevant to TLS:

- (a) **Concatenation / composite.** A single composite public key and a signature $\sigma = \sigma_E \parallel \sigma_P$ under a new algorithm identifier, as in the IETF LAMPS composite ML-DSA specification [15]. Simple, but not backward compatible: a client that does not recognise the composite algorithm cannot verify it. Without further measures, a concatenation is *separable*—an attacker can strip one component and present the other as a stand-alone signature—which composite specifications prevent by binding the algorithm identifier into the signed message [14, 15].
- (b) **Nested / alternative-signature extensions.** A classical certificate that additionally carries a PQ key and PQ signature in non-critical X.509 extensions, so that legacy verifiers ignore them [13]. Backward compatible, at the cost of carrying both sets of objects to every client.
- (c) **Parallel certificate chains.** The server holds a classical and a PQ chain and selects one per client using `signature_algorithms`. Interoperable, but each client verifies only one algorithm, so it is not hybrid in the AND sense.
- (d) **Mixed chains.** Different algorithms at different levels of the chain [23], e.g., a PQ root and intermediate with a classical leaf, trading size against the level at which quantum resistance is achieved.

DAPV applies whenever a single client must verify both a classical and a PQ signature on the same handshake, i.e., to (a) and (b), and to the PQ-signed levels of (d). It does not apply to (c). Our prototype uses a concatenation encoding (Section 6.1); it is not claimed to satisfy the non-separability properties of the composite specification [15], which a production deployment should adopt. We note that DAPV’s own design makes stripping of σ_P yield at most a provisional session, because the client never promotes a session without a valid σ_P .

2.3. TLS 1.3 authentication

TLS 1.3 [28] supports three ways of authenticating a peer.

- (i) **Certificate-based authentication** in a full handshake. After `ServerHello` establishes handshake traffic keys, the server sends, encrypted, `EncryptedExtensions`, `Certificate` (its chain), `CertificateVerify` and `Finished`. The client validates the chain to a trust anchor, which requires verifying the signature on each certificate issued by a CA in the chain, and verifies `CertificateVerify`: a signature, under the leaf key, over 64 bytes of 0x20, the context string “TLS 1.3, server `CertificateVerify`”, a zero byte and the transcript hash of all handshake messages up to and including `Certificate`. The server `Finished` is an HMAC over the transcript up to and including `CertificateVerify`; the client `Finished` is an HMAC over the transcript up to and including the server `Finished`. A failed `CertificateVerify` check must terminate the handshake with a `decrypt_error` alert; an invalid chain with `bad_certificate` or a related alert.
- (ii) **PSK-based authentication** (session resumption or an external PSK). No certificate or signature is exchanged; authentication is inherited from the session that issued the ticket (or from the external key). With 0-RTT, the client may send early data under the PSK before the server’s first flight; such data is not forward secret and is replayable.

- (iii) **Post-handshake client authentication**, in which the server requests a client certificate after the handshake.

The client may also be authenticated with a certificate during the main handshake (mutual TLS), in which case the *server* performs the verification. DAPV applies to certificate-based authentication in full handshakes (i). For (ii) there is no signature to defer, but resumption must be restricted so that it does not inherit a provisional authentication (Section 4.3). In mutual TLS, the same technique could be applied by the server to the client’s hybrid signature; we do not evaluate this case.

Our implementation defers the PQ component of *every* hybrid verification the client performs: the signatures on the leaf and intermediate certificates of the chain and the `CertificateVerify` signature, i.e., three PQ verifications per handshake with a three-certificate chain (root, intermediate, leaf; the root is the client’s trust anchor and is not sent).

3. Related Work

PQ performance in TLS. Paquin, Stebila and Tamvada [19] benchmarked PQ key exchange and signatures in TLS under emulated network conditions and showed that the effect of size dominates that of computation as packet loss and RTT grow. Sikeridis, Kampanakis and Devetsikiotis [17] studied PQ authentication in TLS 1.3 specifically, showing that the added round trip caused by exceeding the TCP initial congestion window is the main source of slowdown and that verification cost is small; they later extended the analysis to TLS and SSH [18]. Sosnowski et al. [29] and Montenegro et al. [20] report evaluation frameworks and results across classical, hybrid and PQ configurations with OpenSSL and OQS. These studies measure costs; none proposes changing *when* verification is scheduled.

Reducing authentication size. KEMTLS replaces handshake signatures by KEM-based implicit authentication [24], with a variant using pre-distributed keys [30] and an IETF draft [25]; Gonzalez and Wiggers evaluated it on embedded devices [31]. Intermediate-certificate suppression [21, 22] and mixed chains [23] reduce the number of large objects transmitted. These approaches attack the dominant, size-related cost and are complementary to DAPV, which attacks the computational one; with them in place, the relative importance of computation increases.

Hybrid signatures. Bindel et al. [13] formalised hybrid signatures for PKI transition and analysed nested and concatenated combinators; Bindel and Hale [14] and the IETF hybrid-signature spectrum document [27] classify separability and backward-compatibility properties; composite ML-DSA is being standardised in the IETF LAMPS working group [15].

Asynchronous and early-data processing in TLS. Deferring or overlapping cryptographic work with the handshake has precedents. OpenSSL’s asynchronous mode and the QTLS framework [32] offload RSA/ECC operations to accelerators and resume the handshake on completion; this overlaps work *across* connections but keeps each connection blocked until its operation completes, whereas DAPV lets a connection proceed before its PQ verification completes. TLS False Start [33] allowed a TLS 1.2 client to send application data before receiving the server `Finished`, accepting a bounded risk in exchange for one round trip; TLS 1.3 0-RTT similarly trades replay protection for latency [28]. DAPV follows the same pattern—accept a weaker guarantee for a bounded time, under explicit restrictions on what may be sent—applied to the PQ component of authentication.

TCP initial window and pacing. The initial congestion window of ten segments was proposed and standardised after large-scale measurements showed that small windows cost web transfers whole round trips [34, 16]; Sikeridis et al. identified it as the cause of the extra round trips of PQ chains [17]. Pacing congestion controllers such as BBR [35] spread a window over the RTT instead of sending it back to back. We turn these observations into a predictive model and quantify how far enlarging the window recovers the cost of PQ chains under paced and unpaced congestion control.

Formal analysis of deferred authentication. Cremers et al. [36] found an attack on an early draft of TLS 1.3 delayed client authentication that arose because the delayed signature was not bound to the full session transcript; the final TLS 1.3 design and its symbolic analysis [37] ensure such binding. In DAPV the deferred signature is part of `CertificateVerify`, which is included in the transcript authenticated by both `Finished` messages before any application data is sent (Section 4.4). A formal model of the provisional state is future work.

Industrial deployment. Google’s CECPQ1 and CECPQ2 experiments [38, 39, 40] and Cloudflare’s experiments [5] concerned PQ key exchange, which is now widely deployed. Deployment of PQ authentication is at an earlier stage and its size cost is the main obstacle discussed [41, 42].

4. DAPV Design

4.1. Design goals

DAPV is designed to (G1) remove the verification of the PQ component from the client’s handshake critical path; (G2) never promote a session to full authentication unless the PQ component has verified, and terminate the session if it fails; (G3) bound what the application may send before promotion; and (G4) require no change to the server, the messages or the wire format, so that a DAPV client interoperates with any server offering the same hybrid credential. Non-goals are reducing handshake size or round trips, and PQ-only deployments.

4.2. Handshake flow

Figure 1 shows a full TLS 1.3 handshake with DAPV on the client. The numbered steps are:

1. **ClientHello**: the client offers key-exchange groups and, in `signature_algorithms`, the hybrid algorithm.
2. **ServerHello**: the server selects the group; both sides derive handshake traffic keys. All following server messages are encrypted.
3. **EncryptedExtensions, Certificate, CertificateVerify, Finished**: the server sends its chain and the hybrid signature $\sigma_E \parallel \sigma_P$ over the transcript, followed by its **Finished**. The transcript hash now covers σ_P .
4. **Classical verification**: the client validates the chain’s classical signatures and verifies σ_E synchronously. On failure it sends the corresponding fatal alert and aborts; no PQ work is scheduled.
5. **Dispatch**: the client submits the job (pk_P, m, σ_P, sid) to a verification worker and enters **PROVISIONAL**. Here m is the **CertificateVerify** content defined in Section 2.3.
6. **Client Finished**: the client verifies the server **Finished**, sends its own **Finished** and derives the application traffic keys. The handshake is complete from the server’s point of view, and application data may flow subject to the policy of Section 4.3.
7. **PQ verification**: the worker verifies σ_P . Δt is the time from step 5 to the end of this step; it consists of queueing and scheduling delay plus the verification itself.
8. **Promotion or termination**: on success the session becomes **FULLY_AUTHENTICATED**; on failure the client sends a fatal **decrypt_error** alert, closes the connection, and discards all state derived from the session, including tickets (Section 4.3).

4.3. Provisional-state semantics

Promotion does not re-key. The transition from **PROVISIONAL** to **FULLY_AUTHENTICATED** changes only a local state variable; no message is exchanged and the traffic keys are not changed. Re-keying is unnecessary because the keys depend only on the (EC)DHE/KEM shared secret and the transcript, both fixed at step 6; the PQ verification adds assurance about *who* holds those keys, not about the keys themselves. If σ_P verifies, the peer with whom the keys were established during Δt is the same peer that is now fully authenticated, so data sent during Δt is retroactively covered by the full hybrid guarantee. If it fails, that data may have been sent to an impostor who defeated only the classical component; this is the exposure bounded in Section 8.2. A TLS **KeyUpdate** after promotion is permissible but adds no authentication.

Data policy. While **PROVISIONAL**, the application may send at most D_{policy} bytes and should send only data it would be willing to reveal to a classical-only authenticated peer (Algorithm 3). Received data may be processed but should not trigger irreversible actions.

Tickets, resumption and 0-RTT. Because PSK resumption inherits the authentication of the issuing session, a **NewSessionTicket** received while **PROVISIONAL** must be held and made usable only on promotion, and discarded on failure. Otherwise an attacker who passed only the classical check could obtain a ticket whose resumptions—including 0-RTT early data—never undergo PQ verification.

Failure. On PQ failure the client sends a fatal **decrypt_error** alert (the alert TLS 1.3 prescribes for a failed **CertificateVerify**), closes the connection, marks the peer’s credential as failed for the lifetime of the process, and reports the failure to the application.

4.4. Transcript binding

The server **Finished** MAC covers **CertificateVerify** and therefore σ_P ; the client verifies it before step 6. An attacker therefore cannot substitute σ_P with a signature from another session, or modify it in transit, without causing the server **Finished** check to fail. Binding guarantees *which* signature is verified later; it does not by itself guarantee that the signature is *valid*, which is precisely what step 7 checks.

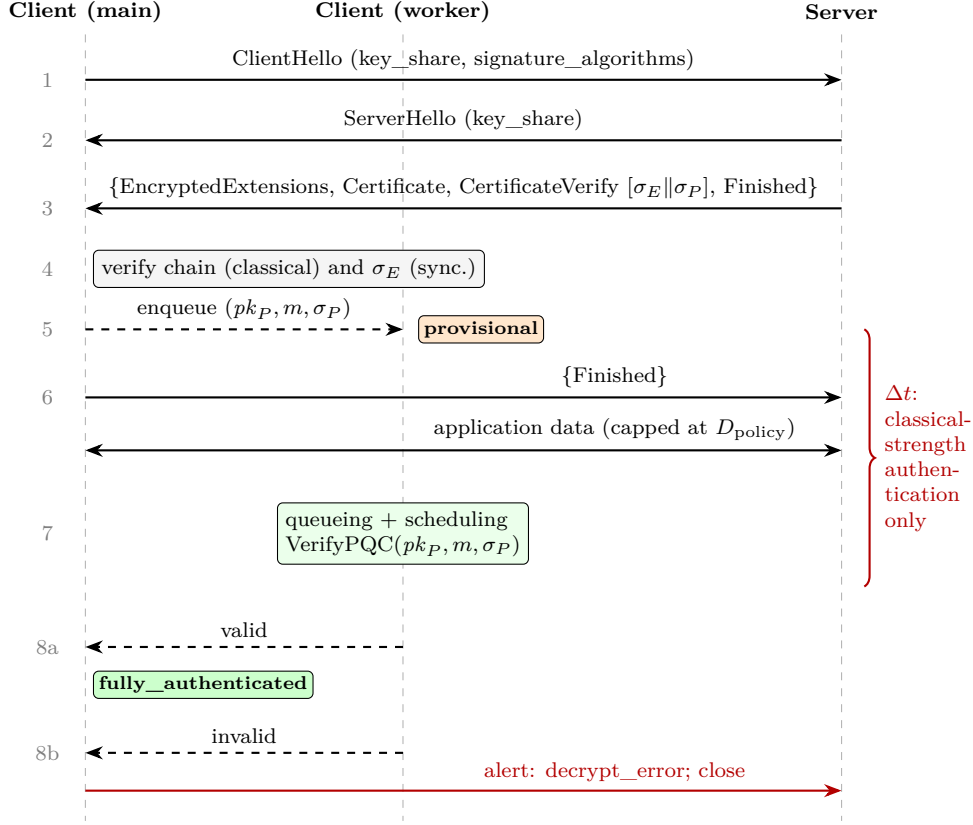


Figure 1: TLS 1.3 full handshake with DAPV on the client. Messages in braces are encrypted under handshake traffic keys. σ_P is part of **CertificateVerify** and hence of the transcript authenticated by both **Finished** messages (step 3 onwards), so the deferred check in step 7 necessarily concerns the signature received in this handshake. No message is added or changed; steps 4–8 are local to the client.

Algorithm 1: DAPV client processing of a hybrid **CertificateVerify**

Input: Leaf keys (pk_E, pk_P) ; message m ; signature $\sigma_E || \sigma_P$; session id sid

```

1 if not VerifyECDSA( $pk_E, m, \sigma_E$ ) then
2   | send alert decrypt_error; abort // no PQ work is scheduled
3 Submit( $pk_P, m, \sigma_P, sid$ ) to the worker pool (Algorithm 2)
4  $session.state \leftarrow$  PROVISIONAL
5 verify server Finished; send client Finished; derive application traffic keys
6 on completion of job ( $sid, ok$ ):
7   | if  $ok$  then
8     |  $session(sid).state \leftarrow$  FULLY_AUTHENTICATED; release held session tickets
9   | else
10  | send alert decrypt_error; close  $session(sid)$ ; discard held tickets

```

4.5. Algorithms

Algorithm 1 gives the client procedure. **VerifyECDSA** and **VerifyPQC** denote the verification algorithms of ECDSA P-256 and of the PQ scheme (ML-DSA.Verify of FIPS 204 [10] or FN-DSA verification [12]), each applied to the corresponding component public key pk_E , pk_P from the leaf certificate, the message m of Section 2.3 and the corresponding signature component; each returns a Boolean.

Dispatch order: after, not concurrently with, the classical check. Two reasons. First, dispatching only after σ_E has verified ensures that a peer who cannot produce a valid classical signature cannot cause any PQ work or occupy a worker; dispatching first would let an unauthenticated attacker consume workers at the cost of a single message, strengthening the flooding attack of Section 8.3. Second, the classical verification takes tens of microseconds, so starting the PQ job earlier would shorten Δt by at most that amount. Implementations that prefer to overlap the two can dispatch first and cancel the job if σ_E fails.

Algorithm 2: Bounded verification worker pool (recommended design; the evaluated with $M = 2$ alongside a thread-per-job variant, Section 6.1)

Input: Pool size M ; queue capacity Q

```

1 start  $M$  worker threads, each executing:
2 while running do
3    $(pk_P, m, \sigma_P, sid) \leftarrow \text{BlockingDequeue}()$ 
4    $ok \leftarrow \text{VerifyPQC}(pk_P, m, \sigma_P)$ 
5   notify the owning connection of  $(sid, ok)$ 

6 function  $\text{Submit}(pk_P, m, \sigma_P, sid)$ :
7   if queue length  $< Q$  then
8     enqueue the job
9   else
10    verify synchronously on the caller's thread           // back-pressure

```

Algorithm 3: Application-layer data policy

Input: Session state; bytes already sent while provisional s ; payload size d ; cap D_{policy}

```

1 if  $state = \text{PROVISIONAL}$  and  $(s + d > D_{\text{policy}})$  or payload is sensitive then
2   buffer payload until FULLY_AUTHENTICATED (or drop on failure)
3 else
4   transmit payload; if  $state = \text{PROVISIONAL}$  then  $s \leftarrow s + d$ 

```

4.6. Design rationale: threads versus instruction-level parallelism

A natural question is why DAPV uses a separate thread when PQ verification is already fast and vector instructions (AVX2, AVX-512) accelerate it further. The two techniques are complementary rather than alternatives. SIMD reduces the *duration* of one verification and is already used by our prototype (liboqs AVX2 code paths). Thread-level deferral changes *where* the remaining duration is spent: off the handshake critical path. For a single AVX2 ML-DSA-44 verification the remaining duration is about $37 \mu\text{s}$, and deferring it cannot save more than that; our measurements confirm that for the lattice schemes the saving is within measurement noise (Section 7.2).

Deferral becomes worthwhile when the synchronous PQ work on the critical path is large, which happens when (i) the chain contains several PQ-signed certificates, possibly with PQ-signed certificate-transparency timestamps, so that a handshake needs several PQ verifications; (ii) a slower scheme such as SLH-DSA is used; (iii) the client lacks vector units (microcontrollers, low-end ARM cores); or (iv) the credential combines more than two algorithms (e.g., ECDSA + ML-DSA + SLH-DSA). Cases (i) and (iv) consist of *independent* verifications, which SIMD cannot parallelise with one another but a worker pool can, one per core. Algorithm 1 generalises directly: all PQ components are submitted as separate jobs and the session is promoted when all have succeeded. Our evaluation covers cases (i) and (ii): three-certificate chains, in which every handshake performs three PQ verifications, and SLH-DSA-128f (Section 7.2); cases (iii) and (iv) remain future work.

5. A Latency Model for Post-Quantum Authentication

We model the client-observed latency of a full TLS 1.3 handshake, from sending `ClientHello` until the client may send application data, as three additive components: RTT-independent processing, verification work on the critical path, and round trips.

5.1. Round trips

Let F be the size in bytes of the server's first flight (`ServerHello` through server `Finished`, including record overhead), W_0 the server's initial congestion window in segments and M the maximum segment size. During slow start the window doubles every round trip, so after n additional round trips the server can have sent $W_0 M (2^{n+1} - 1)$ bytes. The number of *extra* round trips is therefore

$$\delta k(F, W_0) = \min\{n \geq 0 : W_0 M (2^{n+1} - 1) \geq F\}, \quad (1)$$

and the handshake latency is

$$L(\text{RTT}) = L_0 + (1 + \delta k(F, W_0)) \text{RTT}, \quad (2)$$

where L_0 is the latency at zero RTT. Extra round trips arise when the flight exceeds the congestion window, *not* when a message exceeds the MTU: segments within the window are sent back to back. The model has no parameter to fit: F is measured once per configuration, $W_0 = 10$ and $M = 1448$ are Linux defaults [16], and L_0 is measured at zero RTT.

5.2. Verification on the critical path

A synchronous client performs n_v PQ verifications of cost T_v each before it can send its **Finished**; they are part of L_0 . Deferral removes them from the critical path but adds a dispatch cost c (copying the job and waking a worker). The latency gain is

$$\Delta L \approx W_{\text{crit}} - c, \quad W_{\text{crit}} = n_v T_v - W_{\text{hidden}}, \quad (3)$$

where W_{hidden} is verification work that a synchronous client already overlaps with network waiting. With $\delta k = 0$ the whole flight arrives together and $W_{\text{hidden}} = 0$; with $\delta k > 0$ the certificate chain can be verified while later windows are in transit, and only the verifications that depend on the last window (at least **CertificateVerify**) remain on the critical path. Deferral therefore pays off only when $W_{\text{crit}} > c$; since both terms are independent of RTT, the gain is an RTT-independent offset.

5.3. Loss

With independent segment loss probability p , a handshake whose first-flight exchange consists of s data segments is delayed by at least one retransmission round with probability

$$P_{\text{delay}} \approx 1 - (1 - p)^s. \quad (4)$$

Larger PQ flights thus raise the *fraction* of delayed handshakes roughly in proportion to their segment count, even when they need no extra round trip in the loss-free case.

5.4. Predictions for our configurations

The measured first flights (Section 6) are 2.4, 6.1, 12.3, 16.4 and 53.8 KB for P-256, P-256 + FN-DSA-512, P-256 + ML-DSA-44, P-384 + ML-DSA-65 and P-256 + SLH-DSA-128f with three-certificate chains and X25519MLKEM768. Against the default initial window of 14.5 KB, Equation (1) predicts, respectively, $\delta k = 0, 0, 0, 1$ and 2 ; with $W_0 = 20$ it predicts $\delta k = 1$ only for SLH-DSA-128f, and with $W_0 = 40$ no extra round trip at all. Section 7.1 tests these predictions.

6. Implementation and Experimental Setup

6.1. Implementation

We implemented DAPV on OpenSSL 3.5.4, liboqs 0.14.0 (built with AVX2 optimisations) and oqs-provider 0.10.0 [43]. OpenSSL 3.5 provides ML-KEM and the X25519MLKEM768 group natively; the OQS provider supplies the hybrid signature algorithms `p256_mldsa44`, `p256_falcon512`, `p384_mldsa65` and `p256_sphincsha2128fsimple`, which encode a hybrid signature as $\text{len}(\sigma_E) \parallel \sigma_E \parallel \sigma_P$.

Deferral hook. The provider’s hybrid verification routine (`oqs_sig_verify`) first verifies the classical component and then the PQ component. We added 32 lines that, after the classical component has verified, look up a function `dapv_submit` exported by the host process. If it exists and accepts the job, the PQ public key, the signed message and σ_P are copied into a job and the routine returns success; otherwise verification proceeds synchronously, unchanged. Because OpenSSL routes both X.509 chain verification and **CertificateVerify** through this routine, all PQ verifications of a handshake are deferred. The client tags each job with the handshake it belongs to (a thread-local handshake identifier set before each call into the TLS state machine), and a session is fully authenticated when all its jobs have succeeded.

Worker models. *thread*: one new detached thread per job, as in the original prototype; *pool*: M pre-started workers serving a FIFO queue (Algorithm 2 without the synchronous fall-back); we use $M = 2$, one per client core. The synchronous baseline runs the same binaries with deferral disabled.

Correctness. With a test switch that flips one byte of each deferred PQ signature, all 50 of 50 test handshakes were detected as failed PQ verifications after completing provisionally; in all other runs no PQ verification failed.

6.2. Code and data availability

The provider patch, the client and server, the link emulator, the experiment drivers, the raw per-handshake and per-job measurements, and the analysis scripts are available at **[AUTHOR INPUT: public repository URL and archived DOI (e.g., Zenodo)]**.

6.3. Testbed

All experiments ran on one virtual machine with four vCPUs of an Intel Xeon processor at 2.1 GHz (AVX2 and AVX-512), 15 GB of RAM and Linux 6.18 (Ubuntu 24.04, GCC 13.3).

Separating client and server. The server runs in network namespace `srv` pinned to CPU 0, the client in namespace `cli` pinned to CPUs 1–2, and a link emulator on CPU 3. Each namespace has its own TCP/IP stack and a TAP interface with a 1500-byte MTU and all segmentation and receive offloads disabled, so that TCP transmits real 1500-byte frames and runs its own congestion control end to end. The emulator forwards each Ethernet frame between the two TAP devices after a one-way delay of $RTT/2$, serialised at 1 Gbit/s, and can drop frames independently with a configurable probability. (The kernel lacked the `netem` queueing discipline.) Measured ping RTTs are 0.06–0.14 ms with no added delay and 50.3 ms with 50 ms configured. Traffic therefore never uses the loopback interface, and client and server never compete for a core. They still share the memory system and hypervisor of one host (Section 7.7).

TCP. Linux 6.18 defaults: initial congestion window of ten segments, MSS 1448 bytes, and the host’s default congestion control, BBR, which paces transmissions. Experiment E3 additionally uses CUBIC, which does not pace in slow start, selected per route. The size of each configuration’s first flight F is measured once, as the number of TCP payload bytes the client has received when its handshake completes (`tcpi_bytes_received`).

6.4. Configurations

Certificates. For each algorithm we generated a three-certificate chain in which root, intermediate and leaf use the same algorithm; the server sends leaf and intermediate, and the root is the client’s trust anchor. A handshake therefore involves three hybrid verifications. For P-256 + ML-DSA-44 we additionally used a single self-signed leaf trusted directly, matching the original prototype.

Key exchange. X25519MLKEM768 [3] in all runs, so that both confidentiality and authentication are post-quantum; for P-256 + ML-DSA-44 we also ran X25519 to isolate the cost of the hybrid KEM.

Handshakes. TLS 1.3 only, full handshakes (session tickets disabled), `TCP_NODELAY`. After the handshake the client sends a 32-byte request and the server echoes it.

6.5. Experiments and metrics

E1: latency and Δt . Sequential handshakes at RTTs of 0, 20, 50, 100 and 150 ms: 1000 handshakes per configuration at 0 ms and 200 at each other RTT, with all configurations interleaved in ten rounds so that drift of the host affects all equally. *Handshake latency* is measured at the client from sending `ClientHello` to the return of `SSL_connect`, i.e., when the client has sent its `Finished` and may send application data. *Time to full authentication* is measured from `ClientHello` to the completion of the last PQ job (for synchronous verification it equals the handshake latency). For every job we record the queueing delay, the duration of the verification call itself, and Δt , from submission to completion. E1 comprises 21,600 handshakes and 43,200 deferred verifications.

E2: completed-handshake rate. Closed-loop load with 100 or 500 concurrent connections, split over two client processes; each connection performs a handshake and the request–response exchange, then closes and is replaced. Rates are counted over 10 s after 3 s of warm-up, three repetitions per point, at RTTs of 0 and 50 ms. Two CPU placements are compared: *separate* (server on CPU 0, clients on CPUs 1–2) and *co-located* (server and clients share CPUs 0–1, as in the original prototype). We also record server and client CPU utilisation.

E3: initial congestion window. Sequential synchronous handshakes with server initial windows of 10, 20 and 40 segments (with a matching client receive window) at RTTs of 50 and 150 ms, under BBR and CUBIC; 100 handshakes per point, for P-256 and the ML-DSA-44, ML-DSA-65 and SLH-DSA-128f hybrids.

E4: packet loss. Sequential synchronous handshakes at 50 ms RTT with 0, 1 and 3% independent frame loss in each direction; 400 handshakes per point, for P-256 and the FN-DSA-512, ML-DSA-44 and ML-DSA-65 hybrids.

E5: client without vector instructions. As E1 at RTTs of 0 and 50 ms, but the client and the provider it loads use a portable C build of liboqs without AVX2, raising ML-DSA-44 verification from 37 to 115 μ s; the server keeps the AVX2 build.

E6: verification-cost scaling. To locate the break-even point of deferral in a controlled way, each PQ verification is performed k times ($k = 1, 2, \dots, 64$), identically in the synchronous (provider) and deferred (worker) paths, which emulates a verifier k times slower; P-256 + ML-DSA-44 at RTTs of 0 and 50 ms, 300 and 100 handshakes per point. At $k = 64$ one verification costs 2.0 ms, approaching the millisecond-scale verification times reported for microcontrollers [44].

Statistics. For latency we report medians and 95th percentiles; differences between DAPV and synchronous verification are differences of medians, with 95% bootstrap confidence intervals (1000 resamples). For rates we report the mean and standard deviation over repetitions.

Table 2: Accuracy of the latency model (Equations (1) and (2)) against measured median handshake latency at non-zero RTT. No parameter is fitted: F is measured, W_0 and M are configured, and L_0 is the measured latency at zero RTT.

Subset	Points	Round trips correct	Mean abs. error	Max. abs. error
Default window $W_0 = 10$ (E1, E3)	36	36	1.7%	7.7%
All windows, CUBIC (E3)	24	24	1.9%	5.5%
All windows, BBR (E1, E3)	44	44	2.4%	14.9%
All	68	68	2.2%	14.9%

Table 3: Median handshake latency (ms), synchronous versus DAPV with a two-worker pool, and the difference of medians with 95% bootstrap CI. Three-certificate chains, X25519MLKEM768 unless noted. “RTTs” is the fitted slope of latency against RTT (number of round trips).

Configuration	RTT 0 ms			RTT 150 ms		RTTs
	Sync	DAPV	Δ [95% CI]	Sync	Δ [95% CI]	
P-256 (classical)	1.36	—	—	152.4	—	1.01
P-256+FN-DSA-512	3.26	3.23	−0.02 [−0.05, 0.01]	154.6	−0.16 [−0.36, 0.04]	1.01
P-256+ML-DSA-44	2.65	2.66	+0.01 [−0.02, 0.03]	154.0	+0.06 [−0.14, 0.22]	1.01
thread per job		2.80	+0.15 [0.12, 0.17]		+0.26 [0.01, 0.50]	1.01
X25519 key exchange	2.46	2.49	+0.03 [−0.00, 0.06]	153.7	−0.01 [−0.13, 0.12]	1.01
single certificate	2.03	2.06	+0.03 [0.01, 0.05]	153.3	−0.07 [−0.22, 0.08]	1.01
P-384+ML-DSA-65	4.69	4.69	+0.00 [−0.05, 0.04]	309.0	+0.13 [−0.03, 0.28]	2.03
P-256+SLH-DSA-128f	11.74	10.15	−1.59 [−1.68, −1.51]	478.4	−0.61 [−0.71, −0.49]	3.13

7. Results

7.1. Validation of the latency model

Takeaway: the round-trip count of every configuration is predicted correctly from flight size and initial window alone. Across five schemes, RTTs of 20–150 ms, initial windows of 10, 20 and 40 segments and two congestion controllers, the model predicts the number of round trips correctly in all 68 cases and the median latency with a mean absolute error of 2.2% (Table 2). With the default window the measured slopes of latency against RTT are 1.01 round trips for P-256, FN-DSA-512 and ML-DSA-44 (with one or three certificates), 2.03 for ML-DSA-65 and 3.13 for SLH-DSA-128f (Table 3), against predicted values of 1, 1, 1, 2 and 3. The P-256 + ML-DSA-44 flight of 12.3 KB spans nine segments—eight times the MTU—without costing a round trip; the 16.4 KB ML-DSA-65 flight crosses the 14.5 KB window and costs a full extra RTT, 150 ms at 150 ms RTT, which is more than all PQ computation of all configurations combined.

The largest errors occur for the 54 KB SLH-DSA-128f flight under BBR with enlarged windows (up to 15%, or 32 ms). A packet capture shows why: the server sends the first 13 segments at once but then paces the remainder at BBR’s startup rate of roughly twice the window per RTT [35], spreading the flight over about a third of the RTT. The round-trip count is still correct, but part of each window’s transmission time adds to the latency. Under CUBIC, which sends the window back to back, the same configurations are within 5.5% of the model.

7.2. Latency effect of deferral

Takeaway: for the lattice-based schemes, deferral does not reduce latency; it helps only for SLH-DSA. Table 3 and Figure 2 show that for FN-DSA-512, ML-DSA-44 and ML-DSA-65 the difference between DAPV and synchronous verification is at most 0.03 ms at 0 ms RTT and 0.16 ms at 150 ms, and its confidence interval includes zero in every case except the single-certificate configuration, where DAPV is 0.03 ms *slower*. This is expected: three ML-DSA-44 verifications cost about 0.11 ms in total (Section 7.4), comparable to the cost of copying the jobs and waking a worker. Creating a thread per job makes DAPV consistently slower, by 0.15 ms (5.6%) at 0 ms RTT. For SLH-DSA-128f, each verification costs 0.57 ms, and DAPV lowers the median latency by 1.59 ms (13.5%) at 0 ms RTT, close to the 1.7 ms of the three verifications it removes from the critical path.

At non-zero RTT the SLH-DSA saving falls to 0.52–0.69 ms, about one verification. The reason is the extra round trips: the 54 KB server flight needs three congestion windows, so the **Certificate** message is complete, and its two chain signatures are verified synchronously, while the rest of the flight carrying **CertificateVerify** is still in transit. Synchronous chain verification is then already hidden behind network waiting, and deferral can remove only the final **CertificateVerify** verification from the critical path.

DAPV leaves the slopes of latency against RTT unchanged (Table 3), as Equation (3) predicts.

Takeaway: hybrid key exchange adds a small constant. Replacing X25519 with X25519MLKEM768 adds 0.19 ms at 0 ms RTT (2.46 vs 2.65 ms) and no round trip.

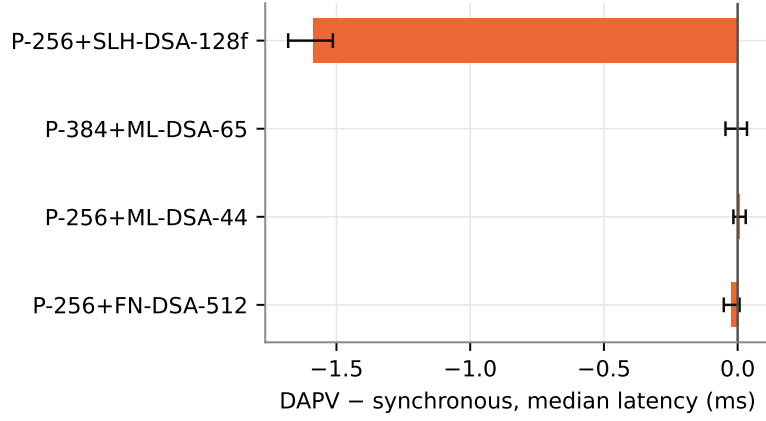


Figure 2: DAPV (pool) minus synchronous median handshake latency at 0 ms RTT, with 95% bootstrap CI (1000 handshakes per configuration). Only SLH-DSA-128f, whose verification costs 0.57 ms, benefits.

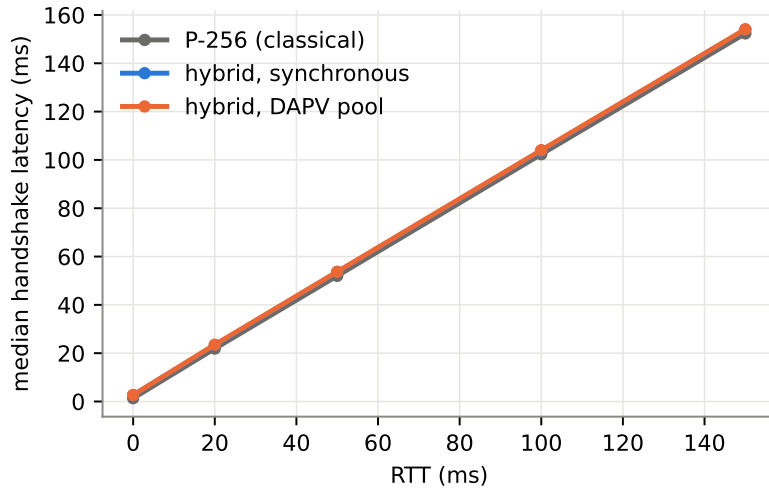


Figure 3: Median handshake latency versus RTT, P-256 + ML-DSA-44 with a three-certificate chain and X25519MLKEM768. The synchronous and DAPV curves coincide; both lie about 1.3 ms above classical P-256 at every RTT (one round trip each).

Hybrid authentication with ML-DSA-44 adds 1.3–1.6 ms to the handshake relative to classical P-256 across RTTs (Figure 3). Of this, client-side PQ verification accounts for only about 0.11 ms; we did not break down the remainder, which comprises server-side signing, transmission of the larger flight and certificate processing.

7.3. When deferral pays: verifier speed

Takeaway: the gain equals the verification work removed from the critical path minus a fixed dispatch cost of about 0.14 ms, exactly as Equation (3) predicts. Without AVX2 (Table 4), ML-DSA verification becomes three times slower, and deferral now helps the lattice schemes too: -0.28 ms (-8.6%) for ML-DSA-44 and -0.48 ms (-9.0%) for ML-DSA-65 at zero RTT, and -3.7 ms (-25.7%) for SLH-DSA-128f. FN-DSA-512 verification needs no vector unit and is unchanged, and so is its (null) gain.

Figure 4 scales the verification cost of ML-DSA-44 in a controlled way. A least-squares fit of the gain against the verification work per handshake W gives $\Delta L = 0.97W - 0.14$ ms ($R^2 = 0.999$) at zero RTT and $\Delta L = 1.03W - 0.23$ ms ($R^2 = 0.998$) at 50 ms: a slope of one, as predicted, and a break-even point at $W^* \approx 0.14$ – 0.23 ms of verification work per handshake. On our AVX2 machine the lattice schemes need 0.11–0.18 ms, just below this point, which explains the null result of Section 7.2. At $k = 64$ (2.0 ms per verification) deferral saves 5.7 ms (-65%) at zero RTT and 6.0 ms at 50 ms.

Takeaway: with several verifications per handshake, deferral also completes authentication sooner. Because the three verifications are independent, two workers run them in parallel. At $k = 64$ the median time to *full* authentication is 6.3 ms with DAPV against 8.8 ms for synchronous verification; at $k = 16$ it is 3.6 against 4.3 ms. Deferral then improves not only the latency seen by the application but also the time until the quantum-safe guarantee holds.

Table 4: DAPV (pool) minus synchronous median handshake latency (ms, 95% bootstrap CI) with a client using a portable liboqs build without AVX2 (E5). “Verify” is the median cost of one PQ verification.

Configuration	Verify	Sync, RTT 0	Δ , RTT 0 ms	Δ , RTT 50 ms
P-256+FN-DSA-512	0.053	3.44	−0.03 [−0.07, 0.01] (−0.8%)	−0.10 [−0.31, 0.14]
P-256+ML-DSA-44	0.115	3.19	−0.28 [−0.31, −0.23] (−8.6%)	−0.23 [−0.49, −0.01]
P-384+ML-DSA-65	0.175	5.36	−0.48 [−0.53, −0.44] (−9.0%)	−0.25 [−0.39, −0.11]
P-256+SLH-DSA-128f	1.714	14.35	−3.68 [−3.76, −3.61] (−25.7%)	−1.26 [−1.39, −1.13]

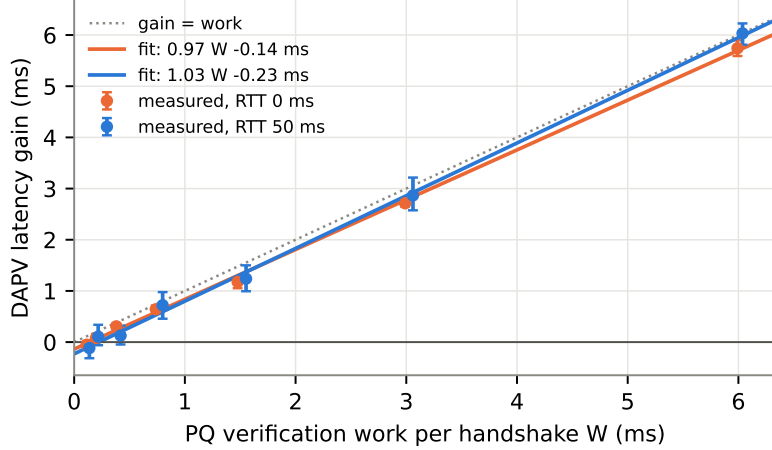


Figure 4: DAPV latency gain versus PQ verification work per handshake (three verifications, each repeated $k = 1 \dots 64$ times), P-256 + ML-DSA-44, with 95% bootstrap CIs and least-squares fits. The dotted line is gain = work.

7.4. Provisional window Δt

Takeaway: with a bounded pool the provisional window is a fraction of a millisecond, and it usually closes before the handshake does. Table 5 and Figure 5 show that for ML-DSA-44 the median Δt is 0.079 ms with a pool and 0.114 ms with a thread per job; the pool also shortens the tail (99th percentile 0.42 versus 0.99 ms, maximum 4.3 versus 9.7 ms). Queueing and wake-up account for about half of Δt ; the verification itself for the rest. Because the client still has to verify the server **Finished**, derive keys and send its own **Finished** after dispatching the jobs, the jobs of the lattice schemes usually finish before **SSL_connect** returns: the median time to full authentication exceeds the median handshake latency by less than 0.01 ms. For SLH-DSA-128f, sessions remain provisional for a median of 0.72 ms after the handshake completes at 0 ms RTT. The rare long tails (up to 36.5 ms) are scheduling delays of the virtual machine. Any data cap must therefore be set against the tail, not the median (Section 8.2).

These values are two orders of magnitude below the 14–21 ms reported for the earlier Windows prototype of this work. Together with the absence of a latency gain for the lattice schemes, this indicates that the earlier measurements reflected costs specific to that prototype, most plausibly timer-quantised waiting on Windows, rather than a property of deferred verification. We therefore no longer report them.

7.5. Completed-handshake rate

Takeaway: DAPV does not increase throughput, with or without co-location. In the separate placement the server core is saturated (utilisation 0.96–0.99) in all configurations except 50 ms RTT with 100 connections, where the rate is bounded by latency. Hybrid authentication lowers the rate from 1935–2045 to 716–721 handshakes/s (by 63–65%) at 0 ms RTT (Table 6, Figure 6). DAPV cannot reduce the server’s work, since it changes only the client, and it does not raise the rate: the pool and thread variants are 1–7% below synchronous verification, within one to two standard deviations. Co-locating client and server lowers all rates, because they now share two cores; the DAPV variants then differ from synchronous verification by −7% to +4%, again within run-to-run variation. The 31–36% gains reported for the earlier prototype are not reproduced.

7.6. Network conditions: initial window, congestion control, loss

Takeaway: a larger initial window recovers the round trips of large PQ chains; pacing retains part of the cost. Figure 7 shows that at 150 ms RTT under CUBIC, raising the server’s initial window from 10 to 20 segments removes the extra round trip of ML-DSA-65 (304 to 156 ms), and raising it to 40 removes both extra round trips of SLH-DSA-128f (454 to 165 ms, −64%). Configurations whose flight already fits in ten segments are unaffected. Under BBR the same changes recover the round trips but the 54 KB SLH-DSA flight remains 2–37 ms slower than under CUBIC (Section 7.1): 190 ms instead of 165 ms at $W_0 = 40$.

Table 5: Per-job verification time and Δt (ms), pooled over all RTTs (three jobs per handshake).

Scheme	Workers	Jobs	Verify (median)	Δt				
				median	p95	p99	p99.9	max
P-256+ML-DSA-44	pool	5400	0.037	0.079	0.21	0.42	1.88	4.3
P-256+ML-DSA-44	thread	5400	0.043	0.114	0.31	0.99	2.75	9.7
P-256+FN-DSA-512	pool	5400	0.055	0.100	0.22	0.48	3.40	6.0
P-384+ML-DSA-65	pool	5400	0.059	0.120	0.27	0.60	3.24	10.2
P-256+SLH-DSA-128f	pool	5400	0.567	0.700	1.71	2.31	3.87	36.5

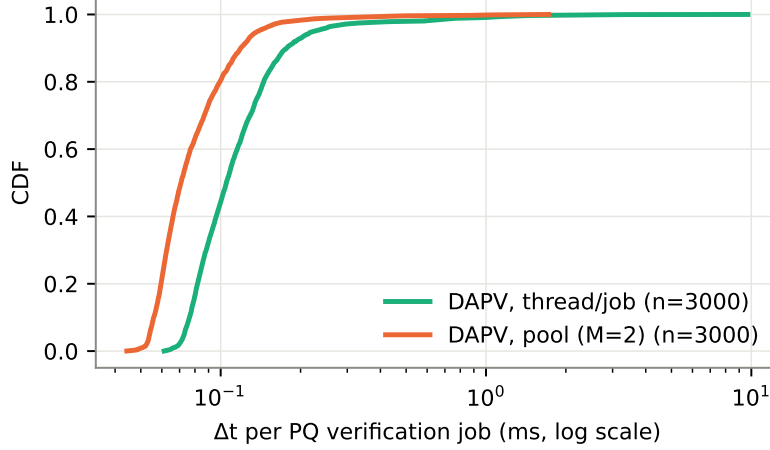


Figure 5: Distribution of Δt for P-256 + ML-DSA-44 at 0 ms RTT (3000 jobs per worker model).

Takeaway: loss penalises PQ handshakes in proportion to their segment count. Loss does not move the median, but it delays a growing fraction of handshakes by at least one retransmission (Table 7, Figure 8). At 3% loss, 8% of classical handshakes are delayed but 16% of FN-DSA-512 and 27% of ML-DSA-44 handshakes, and the 90th percentile of both hybrids more than doubles (to 123–124 ms) while that of P-256 stays at 54 ms. Equation (4) predicts these fractions within 4 percentage points for P-256, FN-DSA-512 and ML-DSA-44. It overestimates them for ML-DSA-65, whose flight already takes two round trips: losses in the first window are often repaired within the extra round trip that the handshake needs anyway.

7.7. Threats to validity

- **One host.** Client and server have separate network stacks and disjoint cores but share the last-level cache, memory bandwidth and hypervisor of one four-vCPU virtual machine; physically separate hosts would remove this residual interference. VM scheduling also causes the rare multi-millisecond Δt outliers.
- **Emulated link.** Delay is added by a userspace emulator with a fixed delay, a 1 Gbit/s rate and no loss; it adds roughly 0.03–0.07 ms per direction. Loss, jitter and bandwidth limits, which would lengthen the critical path further, were not studied.
- **Software stack.** OpenSSL 3.5 spends about 1.4 ms on a classical handshake on this machine; with a faster stack, the relative (not the absolute) SLH-DSA saving would be larger. Results for other TLS libraries may differ.
- **Platform.** One x86-64 CPU with AVX2. Slower verifiers are represented by a portable build of liboqs (E5) and by repeating each verification (E6); the latter reproduces the cost but not other properties (cache sizes, clock speed of the rest of the stack) of embedded processors.
- **Congestion control.** The host default, BBR, is used throughout except in the CUBIC runs of E3; BBR’s pacing adds delay to large flights (Section 7.1).
- **Server concurrency model.** The server uses one thread per connection on a single core; an event-driven server would reach a higher absolute rate, but DAPV does not change the server’s work.

Table 6: Completed handshakes per second (mean $\pm$ s.d. of three 10-s runs), P-256+ML-DSA-44 hybrid unless noted, X25519MLKEM768. Server CPU utilisation is shown for the separate placement.

Placement	RTT / conns	P-256	Sync	DAPV thread	DAPV pool	Server CPU
separate	0 ms / 100	2045 $\pm$ 33	716 $\pm$ 27	670 $\pm$ 48	691 $\pm$ 33	0.96–0.99
separate	0 ms / 500	1935 $\pm$ 73	721 $\pm$ 17	681 $\pm$ 46	670 $\pm$ 47	0.98–0.99
separate	50 ms / 100	646 $\pm$ 4	492 $\pm$ 6	477 $\pm$ 12	474 $\pm$ 14	0.35–0.74
separate	50 ms / 500	1852 $\pm$ 66	672 $\pm$ 23	668 $\pm$ 15	624 $\pm$ 30	0.88–0.99
co-located	0 ms / 100	1327 $\pm$ 13	482 $\pm$ 81	491 $\pm$ 5	500 $\pm$ 7	
co-located	0 ms / 500	1317 $\pm$ 2	512 $\pm$ 19	478 $\pm$ 29	487 $\pm$ 13	
co-located	50 ms / 100	637 $\pm$ 4	444 $\pm$ 14	426 $\pm$ 5	420 $\pm$ 12	
co-located	50 ms / 500	1294 $\pm$ 75	491 $\pm$ 37	459 $\pm$ 9	490 $\pm$ 13	

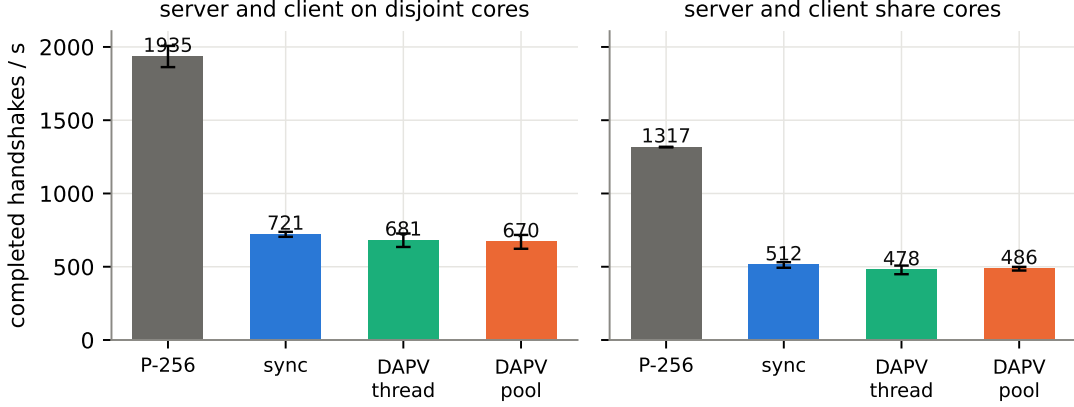


Figure 6: Completed handshakes per second at 0 ms RTT with 500 concurrent connections; error bars are ± 1 s.d. over three runs.

8. Security Analysis

The analysis below is a risk characterisation, not a proof. A formal analysis of the provisional state—e.g., in Tamarin, extending existing TLS 1.3 models [37]—is future work.

8.1. Threat model

The network adversary controls the channel (Dolev–Yao). We distinguish (i) a *classical* adversary, who cannot forge ECDSA or PQ signatures but might hold a fraudulently issued classical credential or exploit a flaw in the classical component, and (ii) a *CRQC* adversary, who can forge ECDSA. Against (i), DAPV provides the same final guarantee as synchronous hybrid verification, but data sent during Δt reaches a peer authenticated only classically. Against (ii), the provisional state offers no protection: DAPV is appropriate only while CRQCs capable of real-time forgery are not a practical threat, and deployments must reassess it as that changes.

8.2. Leakage bound

The data the client can send before a failed PQ check tears down the connection is bounded by the link rate and by the TCP congestion window:

$$L_{\max} = \min\left(\frac{B \Delta t}{8}, W(\Delta t)\right), \quad (5)$$

where B is the bottleneck bandwidth in bit/s and $W(\Delta t)$ is the amount the client’s congestion controller allows in flight during Δt on a new connection (about 14.6 KB per RTT at the initial window [16]). When $\Delta t < \text{RTT}$, $W(\Delta t)$ is at most one initial window, so the exposure is limited to about 14.6 KB irrespective of link speed; the bandwidth term dominates only on low-RTT paths. Table 8 gives the bandwidth term; the application cap D_{policy} bounds both.

In practice the exposure is smaller still. For the lattice schemes the deferred verifications usually complete before the client has finished its own side of the handshake (Section 7.4), so the session is typically already fully authenticated when the application first writes. The cap matters for the tail and for expensive schemes: a scheduling delay like the worst one we observed (36.5 ms) would, on a low-RTT gigabit path, leave room for megabytes without a cap.

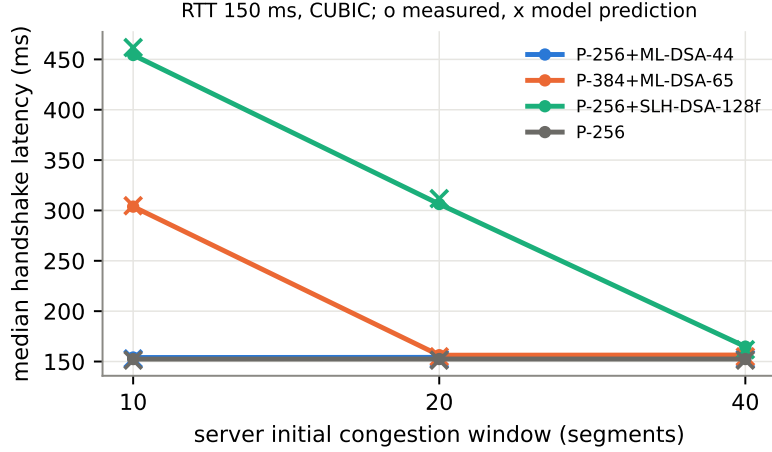


Figure 7: Median handshake latency versus the server’s initial congestion window at 150 ms RTT under CUBIC (synchronous verification, 100 handshakes per point). Circles: measured; crosses: model.

Table 7: Handshake latency (ms) at 50 ms RTT with independent frame loss in each direction (E4; 400 handshakes per point). “Delayed” is the fraction of handshakes more than 25 ms slower than the loss-free median; “model” is Equation (4) with s the server flight’s segments plus the `ClientHello`.

Configuration	Loss	Median	p90	Mean	Delayed	Model
P-256	1%	52.3	53.6	57.8	2.8%	3.0%
	3%	52.2	54.1	71.4	8.0%	8.7%
P-256+FN-DSA-512	1%	54.3	56.1	60.7	6.3%	5.9%
	3%	54.5	123.7	89.2	15.5%	16.7%
P-256+ML-DSA-44	1%	53.9	104.2	66.8	13.0%	9.6%
	3%	54.2	122.6	87.0	26.8%	26.3%
P-384+ML-DSA-65	1%	105.2	107.8	113.3	5.3%	13.1%
	3%	105.9	172.1	135.2	21.5%	34.7%

Choice of D_{policy} . The cap is a policy parameter, not a quantity our measurements determine. Its correct value is the amount of data the application is willing to reveal to a peer that has passed only the classical check, which depends on the content, not on Δt . What Δt determines is how often the cap is reached and hence how often data must be buffered; with the measured Δt this is rare. As a heuristic default we suggest 8 KB: it is below one initial congestion window, so on WAN paths the cap rather than TCP is the binding limit; it exceeds the bandwidth term at the 99th-percentile Δt on links of up to 100 Mbit/s, so it is rarely reached there; and it accommodates typical request headers and small requests. Applications should lower it, or set it to zero, for any content they consider sensitive.

8.3. Attack vectors and mitigations

Impersonation with a valid classical credential. An adversary holding a fraudulently obtained classical key for the server passes step 4 and receives up to L_{max} bytes before step 8 terminates the session. Mitigation: D_{policy} and the restriction to non-sensitive content.

Component stripping. Replacing σ_P with an invalid value causes the server `Finished` check to fail (transcript binding); presenting a classical-only credential is rejected because the client requires the hybrid algorithm. Either way no session is promoted.

Replay. A `CertificateVerify` from another session does not verify against this session’s transcript hash, which includes both parties’ fresh key shares.

Ticket inheritance. Addressed by holding tickets until promotion (Section 4.3).

Worker exhaustion (DAPV-specific). In mutual-TLS or other settings where the verifier accepts many handshakes—e.g., a server verifying hybrid client certificates with DAPV—an adversary can send a valid σ_E with an invalid σ_P on many connections, each occupying a worker until its job fails. With one thread per job this exhausts memory and threads. Mitigations: a bounded pool with bounded queue and synchronous fall-back (Algorithm 2), which degrades DAPV to synchronous verification under attack instead of failing; and per-source rate limiting. Dispatching only after σ_E succeeds (Algorithm 1) means the attack requires a valid classical signature per connection.

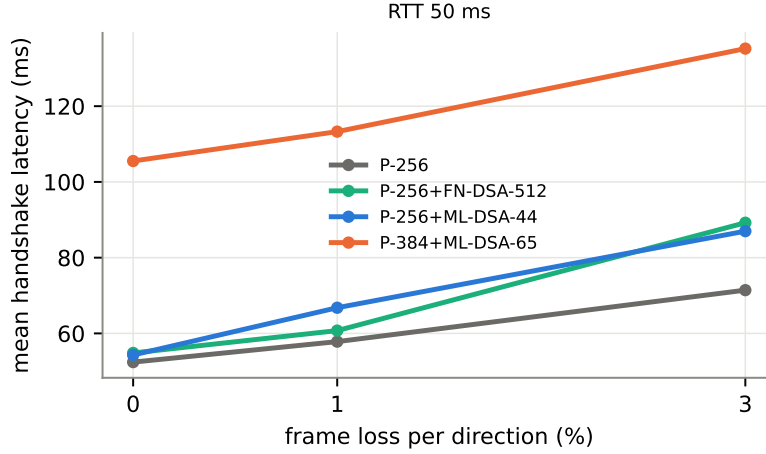


Figure 8: Mean handshake latency versus frame loss at 50 ms RTT.

Table 8: Bandwidth term $B \Delta t / 8$ of Equation (5) for measured Δt values (Table 5; decimal units).

Link rate	ML-DSA-44, pool		SLH-DSA-128f	All runs
	median 0.079 ms	p99 0.42 ms	p99 2.31 ms	max 36.5 ms
1 Mbit/s	10 B	53 B	289 B	4.6 kB
100 Mbit/s	1.0 kB	5.3 kB	29 kB	456 kB
1 Gbit/s	9.9 kB	53 kB	289 kB	4.6 MB
10 Gbit/s	99 kB	525 kB	2.9 MB	46 MB

8.4. Application-layer interface

Enforcing D_{policy} requires the TLS library to expose the state. We propose the following minimal OpenSSL-style interface; it is a design proposal and is not part of the evaluated implementation, whose measurement client tracks the state internally:

```
#define SSL_DAPV_STATE_NONE      0    /* not a DAPV session          */
#define SSL_DAPV_STATE_PROVISIONAL 1  /* sigma_E ok, sigma_P pending */
#define SSL_DAPV_STATE_FULL_AUTH 2    /* sigma_E and sigma_P ok      */
#define SSL_DAPV_STATE_PQC_FAILED 3   /* sigma_P failed; closed      */

int  SSL_get_dapv_state(const SSL *ssl);
void SSL_CTX_set_dapv_callback(SSL_CTX *ctx,
    void (*cb)(SSL *ssl, int new_state, void *arg), void *arg);
```

The callback is invoked on each state change, allowing an application to release buffered data on promotion and to abort pending work on failure; `SSL_write` can enforce D_{policy} by returning a retry-later indication while the session is provisional and the cap is reached.

9. Deployment Guidance

The model and measurements translate into an ordered set of rules. The first three concern size, which dominates; only the fourth concerns deferral.

1. *Keep the first flight within the initial window.* Estimate the server’s first flight F (certificates actually sent, the `CertificateVerify` signature and about 1.4 KB for the other messages with X25519MLKEM768) and compare it with $W_0 \cdot 1448$ bytes. With the default $W_0 = 10$ (14.5 KB), a two-certificate hybrid chain with ML-DSA-44 (12.3 KB) or FN-DSA-512 (6.1 KB) fits; ML-DSA-65 (16.4 KB) does not and costs a full extra RTT; SLH-DSA-128f (53.8 KB) costs two. Every kilobyte beyond the window is far more expensive than any verification.

2. *Otherwise, raise the server’s initial window.* Where the flight cannot be reduced, a larger initial window on the server removes the extra round trips (Section 7.6); large content providers already use windows above ten segments [34]. With a pacing congestion controller such as BBR, part of the cost remains because the window is spread over the RTT.

Table 9: Positioning relative to prior work.

Work	Cost addressed	Approach
[19, 17, 18, 29, 20]	size and computation	measurement; no mitigation
[21, 22, 23]	size (chain)	omit or change certificates in the chain
[24, 30, 25]	size (handshake signature)	KEM-based implicit authentication
[32]	computation (throughput)	asynchronous offload; connection waits for result
This work	size and computation	validated latency model; initial window; defer PQ verification past handshake completion under a provisional state

3. *Reduce what is sent.* Intermediate-certificate suppression [21, 22] and mixed chains [23] shrink F directly; certificate compression [45] helps little, because PQ keys and signatures are close to incompressible; KEM-based authentication [24] removes the handshake signature altogether.

4. *Defer verification only above the break-even point.* Enable DAPV only when the PQ verification work on the client’s critical path exceeds about 0.15–0.25 ms per handshake: SLH-DSA, clients without vector instructions, multi-algorithm credentials, or long chains of expensive signatures. Use a bounded pool, never a thread per job, which is slower than not deferring at all. On AVX2-class hardware with ML-DSA or FN-DSA, synchronous verification is the better choice.

5. *When deferring, enforce the provisional semantics.* Expose the provisional state to applications and enforce a data cap (8 KB by default, zero for sensitive requests); withhold session tickets until promotion; bound the worker pool and queue with synchronous fall-back; and monitor Δt (median and 99th percentile) and the PQ failure rate. Clients that immediately send credentials, payment data or personal data should verify synchronously.

Standardisation. Because DAPV changes no messages, it needs no TLS extension to interoperate. What would benefit from standardisation is guidance on the provisional semantics—ticket handling, permitted data and alerting—for implementations that choose to defer, analogous to the guidance that accompanied TLS False Start [33].

10. Comparison with Prior Work

Table 9 contrasts DAPV with prior work by target and method. Numerical comparison across studies is not meaningful because testbeds, chains and network conditions differ.

11. Conclusion

We set out to explain, predict and reduce the latency that hybrid PQ authentication adds to TLS 1.3 handshakes. A simple model with no fitted parameters—processing, verification on the critical path, and round trips set by the first flight’s size relative to the initial congestion window—predicts every round-trip count we measured and the median latency within a few percent. Its main lesson is that size, not computation, dominates: whether a chain’s first flight fits into the initial window decides whether the handshake takes one, two or three round trips, and each extra round trip outweighs all PQ computation. The most effective mitigations therefore reduce the flight or enlarge the server’s initial window; under a pacing congestion controller the latter recovers most but not all of the cost.

Deferring PQ verification past handshake completion is safe to specify and cheap to implement, but it pays off only above a break-even point of about 0.14–0.23 ms of verification work per handshake. On AVX2 server CPUs, lattice-based hybrids fall below this point; clients without vector instructions, SLH-DSA, multi-signature chains and embedded-class verifiers lie above it, and there deferral cuts latency by 9–65% and, by verifying independent signatures in parallel, also shortens the time to full quantum-safe authentication. The provisional window it creates is a fraction of a millisecond and its exposure is bounded by an application-layer data cap.

Future work

1. Validation on physically separate hosts and real Internet paths, including QUIC, whose congestion control and anti-amplification limit interact with first-flight size differently.
2. Measurements on ARM Cortex-A and microcontroller clients, to place real devices on the break-even curve of Figure 4.
3. Multi-algorithm credentials, where more independent verifications can be parallelised.
4. A formal (e.g., Tamarin) model of the provisional state.
5. Server-side deferral for hybrid client certificates in mutual TLS.

Acknowledgements

The authors thank the Open Quantum Safe project and the OpenSSL community for the open-source infrastructure used in this work.

Declaration of Competing Interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported in this paper.

References

- [1] P. W. Shor, Algorithms for quantum computation: Discrete logarithms and factoring, Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS) (1994) 124–134doi:10.1109/SFCS.1994.365700.
- [2] N. Alnahawi, J. Müller, J. Oupicky, A. Wiesmaier, A comprehensive survey on post-quantum TLS, IACR Communications in Cryptology 1 (2) (2024) 41–71.
- [3] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, D. Stebila, Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3, IETF Internet-Draft [draft-ietf-tls-ecdhe-mlkem](#), work in progress (2026).
- [4] National Institute of Standards and Technology, FIPS 203: Module-lattice-based key-encapsulation mechanism standard (MLKEM), Tech. rep., NIST (2024). doi:10.6028/NIST.FIPS.203.
- [5] K. Kwiatkowski, L. Valenta, The TLS post-quantum experiment, Cloudflare Blog, <https://blog.cloudflare.com/the-tls-post-quantum-experiment/> (Oct. 2019).
- [6] M. Mosca, Cybersecurity in an era with quantum computers: Will we be ready?, IEEE Security & Privacy 16 (5) (2018) 38–41. doi:10.1109/MSP.2018.3761723.
- [7] U.S. National Security Agency (NSA), Commercial national security algorithm suite and quantum computing roadmap, Tech. rep., NSA, <https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/3215760> (2022).
- [8] European Union Agency for Cybersecurity (ENISA), Post-quantum cryptography: Preparing for the quantum era, Tech. rep., ENISA, <https://www.enisa.europa.eu/publications/post-quantum-cryptography> (2023).
- [9] U.K. National Cyber Security Centre (NCSC), Next steps in preparing for post-quantum cryptography, Tech. rep., NCSC, <https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography> (2024).
- [10] National Institute of Standards and Technology, FIPS 204: Module-lattice-based digital signature standard (ML-DSA), Tech. rep., NIST, <https://doi.org/10.6028/NIST.FIPS.204> (2024).
- [11] National Institute of Standards and Technology, FIPS 205: Stateless hash-based digital signature standard (SLH-DSA / sphincs+), Tech. rep., NIST, <https://doi.org/10.6028/NIST.FIPS.205> (2024).
- [12] National Institute of Standards and Technology, FIPS 206 (initial public draft): FFT over NTRU-lattice-based digital signature algorithm (FN-DSA), Tech. rep., NIST, draft standard. AUTHORS: verify status, date and URL before submission. (2025).
- [13] N. Bindel, U. Herath, M. McKague, D. Stebila, Transitioning to a quantum-resistant public key infrastructure, in: T. Lange, T. Takagi (Eds.), Post-Quantum Cryptography (PQCrypto 2017), Vol. 10346 of Lecture Notes in Computer Science, Springer, Cham, 2017, pp. 384–405, provides nesting-based hybrid signature construction with composable unforgeability guarantees; differs from the concatenation construction used in our prototype.
- [14] N. Bindel, B. Hale, A note on hybrid signature schemes, Cryptology ePrint Archive, Paper 2023/423, <https://eprint.iacr.org/2023/423> (2023).
- [15] M. Ounsworth, J. Gray, M. Pala, J. Klaussner, S. Fluhrer, Composite ML-DSA for use in X.509 public key infrastructure, IETF Internet-Draft [draft-ietf-lamps-pq-composite-sigs](#), work in progress (2026).
- [16] J. Chu, N. Dukkupati, Y. Cheng, M. Mathis, Increasing TCP’s initial window, RFC 6928, IETF (2013). doi:10.17487/RFC6928.
- [17] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Post-quantum authentication in TLS 1.3: A performance study, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2020. doi:10.14722/ndss.2020.24203.
- [18] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th International Conference on Emerging Networking Experiments and Technologies (CoNEXT’20), ACM, 2020, pp. 149–156. doi:10.1145/3386367.3431305.
- [19] C. Paquin, D. Stebila, G. Tamvada, Benchmarking post-quantum cryptography in TLS, in: J. Ding, J.-P. Tillich (Eds.), Post-Quantum Cryptography, Vol. 12100 of Lecture Notes in Computer Science, Springer, Cham, 2020, pp. 72–91. doi:10.1007/978-3-030-44223-1_5.
- [20] J. A. Montenegro, R. Rios, J. Lopez-Cerezo, A performance evaluation framework for post-quantum TLS, Future Generation Computer Systems 175 (2026) 108062. doi:10.1016/j.future.2025.108062.
- [21] P. Kampanakis, M. Kallitsis, Faster post-quantum TLS handshakes without intermediate CA certificates, in: Cyber Security, Cryptology, and Machine Learning (CSCML 2022), Vol. 13301 of Lecture Notes in Computer Science, Springer, 2022. doi:10.1007/978-3-031-07689-3_25.
- [22] D. Sikeridis, S. Huntley, D. Ott, M. Devetsikiotis, Intermediate certificate suppression in post-quantum TLS: An approximate membership querying approach, in: Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies (CoNEXT), ACM, 2022. doi:10.1145/3555050.3569127.
- [23] S. Paul, Y. Kuzovkova, N. Lahr, R. Niederhagen, Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3, in: Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS), 2022, pp. 727–740. doi:10.1145/3488932.3497755.
- [24] P. Schwabe, D. Stebila, T. Wiggers, Post-quantum TLS without handshake signatures, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2020, pp. 1461–1480. doi:10.1145/3372297.3423350.
- [25] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, N. Sullivan, KEM-based authentication for TLS 1.3, IETF Internet-Draft [draft-celi-wiggers-tls-authkem](#), <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/> (Work in Progress). This draft specifies KEM-based authentication; hybrid signature design is covered by separate IETF work items. (2024).
- [26] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé, CRYSTALS-Dilithium: A lattice-based digital signature scheme, IACR Transactions on Cryptographic Hardware and Embedded Systems 2018 (1) (2018) 238–268. doi:10.13154/tches.v2018.i1.238-268.

- [27] N. Bindel, B. Hale, D. Connolly, F. Driscoll, Hybrid signature spectrums, IETF Internet-Draft [draft-ietf-pquip-hybrid-signature-spectrums](#), work in progress. AUTHORS: verify author list and year.
- [28] E. Rescorla, The transport layer security (TLS) protocol version 1.3, RFC 8446, IETF (2018). doi:10.17487/RFC8446.
- [29] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoiniakakis, S. Gallenmüller, G. Carle, The performance of post-quantum TLS 1.3, in: Companion of the 19th International Conference on Emerging Networking Experiments and Technologies (CoNEXT Companion '23), ACM, 2023, doi = 10.1145/3624354.3630585.
- [30] P. Schwabe, D. Stebila, T. Wiggers, More efficient post-quantum KEMTLS with pre-distributed public keys, in: Computer Security – ESORICS 2021, Vol. 12972 of Lecture Notes in Computer Science, Springer, 2021, pp. 3–22. doi:10.1007/978-3-030-88418-5_1.
- [31] R. Gonzalez, T. Wiggers, KEMTLS vs. post-quantum TLS: Performance on embedded systems, in: Proceedings of the International Conference on Security, Privacy and Applied Cryptographic Engineering (SPACE), 2022, <https://kemtls.org/publication/kemtls-embedded/>.
- [32] X. Hu, C. Wei, J. Li, B. Will, P. Yu, L. Gong, H. Guan, QTLS: High-performance TLS asynchronous offload framework with Intel QuickAssist technology, in: Proceedings of the 24th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), 2019, pp. 158–172. doi:10.1145/3293883.3295705.
- [33] A. Langley, N. Modadugu, B. Moeller, Transport layer security (TLS) false start, RFC 7918, IETF (2016). doi:10.17487/RFC7918.
- [34] N. Dukkipati, T. Refice, Y. Cheng, J. Chu, T. Herbert, A. Agarwal, A. Jain, N. Sutin, An argument for increasing TCP’s initial congestion window, ACM SIGCOMM Computer Communication Review 40 (3) (2010) 26–33. doi:10.1145/1823844.1823848.
- [35] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, V. Jacobson, BBR: Congestion-based congestion control, ACM Queue 14 (5) (2016) 20–53. doi:10.1145/3012426.3022184.
- [36] C. Cremers, M. Horvat, S. Scott, T. van der Merwe, Automated analysis and verification of TLS 1.3: 0-RTT, resumption and delayed authentication, in: Proceedings of the IEEE Symposium on Security and Privacy (S&P), 2016, pp. 470–485. doi:10.1109/SP.2016.35.
- [37] C. Cremers, M. Horvat, J. Hoyland, S. Scott, T. van der Merwe, A comprehensive symbolic analysis of TLS 1.3, in: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1773–1788. doi:10.1145/3133956.3134063.
- [38] M. Braithwaite, Experimenting with post-quantum cryptography, Google Security Blog, <https://security.googleblog.com/2016/07/experimenting-with-post-quantum.html> (Jul. 2016).
- [39] A. Langley, CECPQ1 results, Imperial Violet Blog, <https://www.imperialviolet.org/2016/11/28/cecpq1.html> (Nov. 2016).
- [40] A. Langley, CECPQ2, Imperial Violet Blog, <https://www.imperialviolet.org/2018/12/12/cecpq2.html> (Dec. 2018).
- [41] A. Langley, Quantum-resistant authentication in TLS, Chromium Blog, <https://blog.chromium.org/2023/09/quantum-resistant-authentication-in-tls.html> (Sep. 2023).
- [42] T. Reddy, H. Tschofenig, Post-quantum cryptography recommendations for TLS-based applications, IETF Internet-Draft [draft-ietf-uta-pqc-app-00](#), <https://datatracker.ietf.org/doc/draft-ietf-uta-pqc-app/> (Work in Progress) (Sep. 2025).
- [43] D. Stebila, M. Mosca, Post-quantum key exchange for the internet and the open quantum safe project, in: R. Avanzi, H. Heys (Eds.), Selected Areas in Cryptography (SAC 2016), Vol. 10532 of Lecture Notes in Computer Science, Springer, Cham, 2017, pp. 1–24, oQS benchmarks available at <https://openquantumsafe.org/>.
- [44] M. J. Kannwischer, J. Rijneveld, P. Schwabe, K. Stoffelen, pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4, in: Second NIST PQC Standardization Conference, 2019.
- [45] A. Ghedini, V. Vasiliev, TLS certificate compression, RFC 8879, IETF (2020). doi:10.17487/RFC8879.